20. Documentation, APIs, and Semantic Versioning

20. Documentation, APIs, and Semantic Versioning

Phase: Phase 3 — Idioms and engineering judgment
Estimated time: 1 day
Slug: `20-docs-apis-semver`

Learning objectives

- Write Go doc comments that godoc renders
- Design stable exported APIs
- Apply semver and Go modules major versions
- Deprecate thoughtfully

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Docs and semver are how libraries age gracefully. Go doc comments and module versions are part of the public API.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Doc comments precede declarations and start with the name.
- `go doc` / pkg.go.dev render those comments.
- Semver: major for breaking changes; /v2+ in module paths.
- Compatibility guidelines: don't break callers casually.
- Examples in _test.go appear on pkg.go.dev.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-documentatio && cd /tmp/gw-documentatio
go mod init example.com/scratch
```

Then create `main.go`:

```
package greeter
```

```go
// Hello returns a greeting for name.
// Empty name is allowed and produces "Hello, ".
func Hello(name string) string {
    return "Hello, " + name
}
```

Run it:

```
go run .
```

Expected output:

```
go doc → documents Hello
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Add a doc comment and run `go doc`.
2. Write an ExampleHello test function.
3. Skim Go module versioning docs for /v2.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Doc comments

```go
// Package store implements durable persistence for users.
package store

// Find returns a user by ID.
// It returns ErrNotFound if no user exists.
func Find(ctx context.Context, id string) (*User, error)
```

Start with the name of the symbol. Run `go doc` locally.

Semver

- MAJOR: breaking changes (import path `/v2` for v2+)
- MINOR: compatible new features
- PATCH: bug fixes

API design

- Accept interfaces, return structs (often)
- Keep exported surface small
- Prefer options structs over endless parameters

Failure modes to watch while coding

- Breaking APIs without a major version
- Empty doc comments that restate the name only poorly
- Exporting types “for later”

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- docstrings vs Go doc comments — placement and tooling differ.
- CalVer vs SemVer — Go modules expect Semver.
- Type hints as docs ≠ compile-enforced API stability.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Treat exported names as forever once tagged v1. Prefer additive changes.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Documentation, APIs, and Semantic Versioning” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Breaking APIs without a major version
- Empty doc comments that restate the name only poorly
- Exporting types “for later”

Exercises

Work in `exercises/20-docs-apis-semver/`. Read that folder’s `README.md` before you code.

What you will build

Satisfy the docs/comment exercise — often about exporting and commenting correctly.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/20-docs-apis-semver/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What triggers a major version in Go modules?
2. How should a doc comment start?
3. Why keep the export surface small?

Further reading

- https://go.dev/doc/comment
- https://go.dev/doc/modules/release-workflow